

Tkinter по задачам контекста

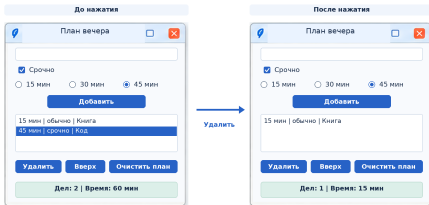
Новая теория ровно тогда, когда она
понадобится

20 задач Tkinter · в вашем контексте №9–28

Дмитрий Нефедов

20 План вечера

Каждое дело хранит своё время и приоритет.



Удалится дело на 45 минут. В плане остаётся 15 минут; настройки нового дела сохранятся.

Слева интерфейс, на плоский экран. Цвета, шрифты и отступы можно выбирать.

Главный принцип

Не изучаем весь Tkinter заранее.
Перед каждой задачей добавляем
только один следующий инструмент.

Как работать с этой презентацией

1 · Открыли задачу

Сначала покажите её схему и сформулируйте, чем она отличается от предыдущей.

2 · Дали один новый инструмент

Покажите следующий слайд с минимальным API и нейтральным примером.

3 · Сразу решаем

Студенты уходят в Контест. Не показывайте материал следующих задач заранее.

4 · Помощь вопросами

Если застряли, возвращайтесь к вопросу и типичной ошибке на слайде, а не к готовому решению.

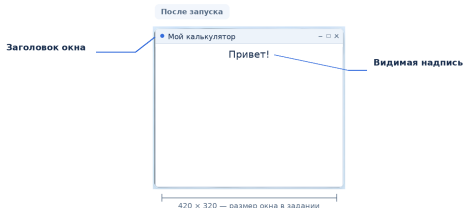
Нумерация: Tkinter 1–20 соответствует задачам 9–28 в вашем текущем конкурсе после восьми задач на функции.

Задача 1: Первое окно

Tkinter 1/20 · в вашем контексте №9

01 Первое окно

Ваше первое настоящее окно — без кнопок и вычислений.



Надпись нужно не только создать, но и разместить. Окно остаётся открытым.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Пока ничего: стартуем с нуля.

Новое перед этой задачей

Tk, Label, title, geometry, pack, mainloop.

Цель решения

Собрать одно окно, одну надпись и запустить цикл событий.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 1: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
import tkinter as tk

root = tk.Tk()
root.title("Учебное окно")
root.geometry("480x280")

msg = tk.Label(root, text="Готово к старту")
msg.pack(pady=12)

root.mainloop()
```

Запомнить

Tk, Label, title, geometry, pack, mainloop.

Создать виджет недостаточно: его нужно разместить через pack/grid.

Перед кодом. Что выполнится один раз при запуске, а что будет поддерживаться mainloop?

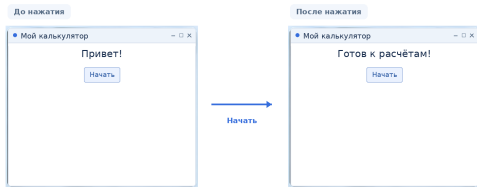
После ответа на вопрос — переходите в Контекст и решайте задачу 9. Следующую теорию показывайте только после попытки.

Задача 2: На связи!

Tkinter 2/20 · в вашем контексте №10

02 На связи!

Одна кнопка меняет сообщение в уже открытом окне.



Повторное нажатие не добавляет новую надпись и не открывает ещё одно окно.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Окно и Label из задачи 1.

Новое перед этой задачей

Button, command, изменение Label.

Цель решения

Добавить кнопку и изменить уже существующую надпись после клика.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 2: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def signal():  
    msg.config(text="Ответ получен")  
  
btn = tk.Button(  
    root,  
    text="Проверить",  
    command=signal  
)  
btn.pack()
```

Запомнить

Button, command, изменение Label.

В command передают функцию без скобок: `command=signal`, а не `command=signal()`.

Перед кодом. Почему функция не должна выполняться в момент создания кнопки?

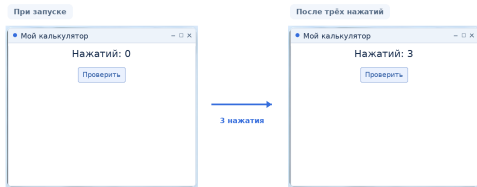
После ответа на вопрос — переходите в Контекст и решайте задачу 10. Следующую теорию показывайте только после попытки.

Задача 3: Пульс кнопки

Tkinter 3/20 · в вашем контексте №11

03 Пульс кнопки

У приложения появляется состояние, которое переживает отдельный клик.



Каждое нажатие прибавляет единицу. Новый запуск начинается отсчёт с нуля.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Кнопка уже умеет вызывать обработчик.

Новое перед этой задачей

состояние между кликами.

Цель решения

Сохранить счётчик между отдельными вызовами обработчика.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 3: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
count = 0

def click():
    global count
    count += 1
    msg.config(text=f"Событий: {count}")
```

Запомнить

состояние между кликами.

Если обнулить count внутри функции, каждый клик начнётся заново.

Перед кодом. Где должна жить переменная, чтобы второй клик видел результат первого?

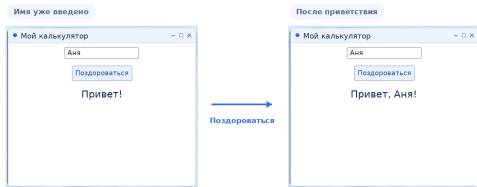
После ответа на вопрос — переходите в Контекст и решайте задачу 11. Следующую теорию показывайте только после попытки.

Задача 4: Калькулятор знакомится

Tkinter 4/20 · в вашем контексте №12

04 Калькулятор знакомится

Поле ввода превращает приветствие в личное обращение.



Имя читается в момент нажатия. Пустое имя превращается в обращение «гость».

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Умеем обрабатывать нажатие кнопки.

Новое перед этой задачей

`Entry.get()`, `strip()`, пустая строка.

Цель решения

Прочитать актуальный текст поля именно в момент клика.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 4: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def show_name():  
    name = entry.get().strip()  
    if name:  
        msg.config(text=f"Здравствуйтесь, {name}!")  
    else:  
        msg.config(text="Имя не указано")
```

Запомнить

Entry.get(), strip(), пустая строка.

Не считывайте Entry при запуске:
тогда пользователь ещё ничего не
ввёл.

Перед кодом. Почему entry.get()
нужно вызывать внутри обработ-
чика?

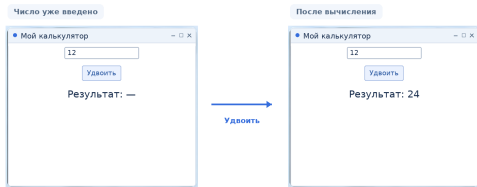
После ответа на вопрос — перехо-
дите в Контекст и решайте задачу 12.
Следующую теорию показывайте
только после попытки.

Задача 5: Удвоитель планов

Tkinter 5/20 · в вашем контексте №13

05 Удвоитель планов

Текст из поля становится числом, а число — результатом вычисления.



Новый клик снова удваивает число из поля, а не предыдущий результат.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Умеем получать строку из Entry.

Новое перед этой задачей

`int()`, вычисление по введённому числу.

Цель решения

Преобразовать строку в число и показать числовой результат.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 5: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def calc():  
    x = int(entry.get())  
    y = x * 3  
    msg.config(text=f"Ответ: {y}")
```

Запомнить

`int()`, вычисление по введённому числу.

Entry всегда возвращает строку, даже если на экране написано 42.

Перед кодом. Что будет типом `entry.get()` после ввода 42?

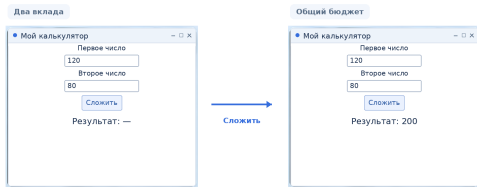
После ответа на вопрос — переходите в Контекст и решайте задачу 13. Следующую теорию показывайте только после попытки.

Задача 6: Бюджет на двоих

Tkinter 6/20 · в вашем контексте №14

06 Бюджет на двоих

Два поля и один результат: собираем основу калькулятора.



Сначала ввод, потом нажатие. Одно редактирование поля не запускает расчёт.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Умеем читать одно число и считать.

Новое перед этой задачей

два Entry и сбор данных формы.

Цель решения

Прочитать два независимых поля и выполнить один расчёт.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 6: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def calc():  
    a = int(first.get())  
    b = int(second.get())  
    msg.config(text=f"Ответ: {a + b}")
```

Запомнить

два Entry и сбор данных формы.

Считывайте оба поля заново при каждом клике.

Перед кодом. Какие значения должны быть прочитаны, если пользователь исправил только второе поле?

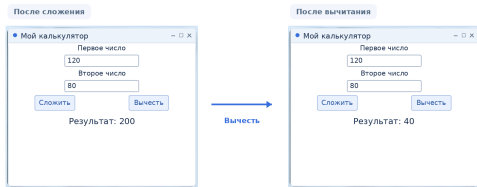
После ответа на вопрос — переходите в Контекст и решайте задачу 14. Следующую теорию показывайте только после попытки.

Задача 7: Плюс или минус

Tkinter 7/20 · в вашем контексте №15

07 Плюс или минус

Одни и те же данные — две разные операции.



Вычитаем второе число из первого. Поля при вычислении не меняются.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Два поля уже есть.

Новое перед этой задачей
несколько callback в одном интерфейсе.

Цель решения

Разделить два действия: одни и те же данные, разные обработчики.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 7: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def add():  
    ...  
  
def subtract():  
    ...  
  
tk.Button(root, text="+", command=add)  
tk.Button(root, text="-", command=subtract)
```

Запомнить

несколько callback в одном интерфейсе.

Не храните старый результат как источник для нового вычисления.

Перед кодом. Что общего у двух кнопок и что у них должно различаться?

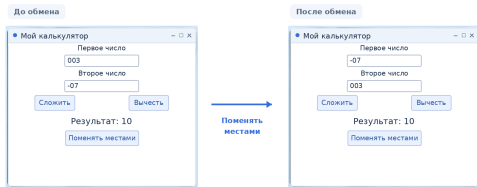
После ответа на вопрос — переходите в Контекст и решайте задачу 15. Следующую теорию показывайте только после попытки.

Задача 8: Рокировка чисел

Tkinter 8/20 · в вашем контексте №16

08 Рокировка чисел

Меняю порядок аргументов, не перепечатывая их вручную.



Обмениваются именно строки: «003» остаётся «003». Результат не пересчитывается.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Умеем читать оба Entry.

Новое перед этой задачей

Entry.delete(), Entry.insert(), обмен строк.

Цель решения

Сначала сохранить обе строки, потом очистить и вставить их в обратном порядке.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 8: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
a = first.get()
b = second.get()

first.delete(0, tk.END)
second.delete(0, tk.END)
first.insert(0, b)
second.insert(0, a)
```

Запомнить

Entry.delete(), Entry.insert(), обмен строк.

Менять нужно строки как есть: 003 должно остаться 003.

Перед кодом. Почему при обмене нельзя сначала преобразовывать значения в int?

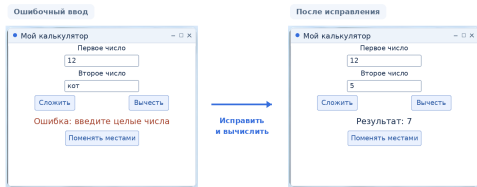
После ответа на вопрос — переходите в Контекст и решайте задачу 16. Следующую теорию показывайте только после попытки.

Задача 9: Право на ошибку

Tkinter 9/20 · в вашем контексте №17

09 Право на ошибку

Показываем понятную ошибку и даём пользователю исправиться.



Ошибка не закрывает окно. Исправленный ввод позволяет продолжить работу.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Арифметика и обмен уже работают.

Новое перед этой задачей

try/except ValueError.

Цель решения

Защитить только числовые операции и позволить пользователю исправить ввод.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 9: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def calc():  
    try:  
        a = int(first.get())  
        b = int(second.get())  
        msg.config(text=f"Ответ: {a + b}")  
    except ValueError:  
        msg.config(text="Введите целые числа")
```

Запомнить

try/except ValueError.

Обмен строк не требует int() и не должен ломаться на слове или пустой строке.

Перед кодом. Какая операция здесь действительно может вызвать ValueError?

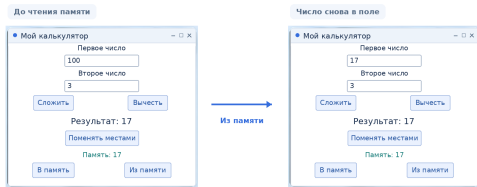
После ответа на вопрос — переходите в Контекст и решайте задачу 17. Следующую теорию показывайте только после попытки.

Задача 10: Карманная память

Tkinter 10/20 · в вашем контексте №18

10 Карманная память

Сохраняем промежуточное число и возвращаем его в следующий расчёт.



Чтение памяти меняет только первое поле. Второе поле и показанный результат сохраняются.

Схема интерфейса. Цвета, шрифты и отступы можно выбирать самостоятельно.

Уже умеем

Есть результат, поля и обработка ошибок.

Новое перед этой задачей

отдельное состояние; None и 0.

Цель решения

Отделить сохранённую память от текущего результата и содержимого полей.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 10: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
memory = None
last_result = None

def save():
    global memory
    if last_result is not None:
        memory = last_result
```

Запомнить

отдельное состояние; None и 0.

0 - корректное сохранённое число, поэтому «пусто» лучше обозначать None.

Перед кодом. Почему memory = 0 не подходит как единственный признак пустой памяти?

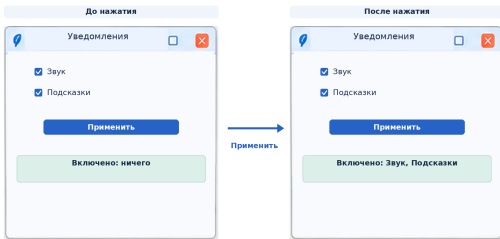
После ответа на вопрос — переходите в Контекст и решайте задачу 18. Следующую теорию показывайте только после попытки.

Задача 11: Тишина по запросу

Tkinter 11/20 · в вашем контексте №19

11 Тишина по запросу

Две независимые опции — одно подтверждение.



Переключение флажков не меняет последнее подтверждённое сообщение.

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Состояние уже знакомо на обычных переменных.

Новое перед этой задачей

Checkbox + BooleanVar.

Цель решения

Получать несколько независимых значений да/нет из интерфейса.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 11: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
sound = tk.BooleanVar(value=False)

box = tk.Checkbutton(
    root,
    text="Звук",
    variable=sound
)
box.pack()

# текущее состояние
print(sound.get())
```

Запомнить

Checkbutton + BooleanVar.

Для независимых флажков нужны независимые переменные.

Перед кодом. Что произойдёт, если два Checkbutton привязать к одной BooleanVar?

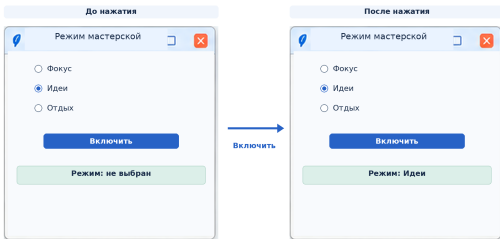
После ответа на вопрос — переходите в Контекст и решайте задачу 19. Следующую теорию показывайте только после попытки.

Задача 12: Режим мастерской

Tkinter 12/20 · в вашем контексте №20

12 Режим мастерской

Один режим из трёх. Выбор и подтверждение — разные действия.



В группе всегда выбран один вариант. Повторный выбор не снимает отметку.

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Знаем Tkinter-переменные.

Новое перед этой задачей

Radiobutton + общая переменная.

Цель решения

Организовать выбор ровно одного режима из нескольких.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 12: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
mode = tk.StringVar(value="basic")

tk.Radiobutton(root, text="Базовый",
               variable=mode, value="basic").pack()
tk.Radiobutton(root, text="Про",
               variable=mode, value="pro").pack()

print(mode.get())
```

Запомнить

Radiobutton + общая переменная.

У группы одна variable, но разные value.

Перед кодом. Почему у радиокнопок одной группы переменная общая?

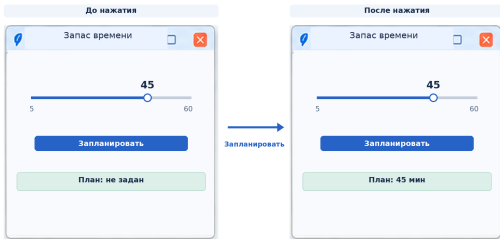
После ответа на вопрос — переходите в Контекст и решайте задачу 20. Следующую теорию показывайте только после попытки.

Задача 13: Запас времени

Tkinter 13/20 · в вашем контексте №21

13 Запас времени

Выберите длительность от 5 до 60 минут с шагом 5.



Ползунок уже на 45. Надпись изменится только после кнопки «Запланировать».

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Умеем читать состояние элементов управления.

Новое перед этой задачей

Scale: диапазон, шаг, get/set.

Цель решения

Получить число с ползунка только тогда, когда это требуется действием.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 13: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
level = tk.Scale(  
    root,  
    from_=0,  
    to=100,  
    resolution=10,  
    orient="horizontal"  
)  
level.set(50)  
print(level.get())
```

Запомнить

Scale: диапазон, шаг, get/set.

Параметр называется `from_`, потому что `from` - ключевое слово Python.

Перед кодом. Что меняется сразу при движении ползунка, а что можно менять только по кнопке?

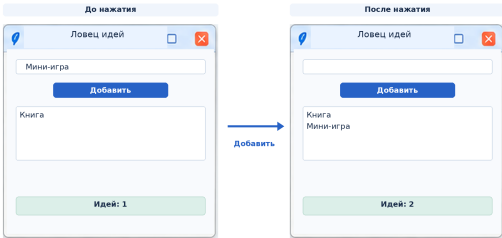
После ответа на вопрос — переходите в Контекст и решайте задачу 21. Следующую теорию показывайте только после попытки.

Задача 14: Ловец идей

Tkinter 14/20 · в вашем контексте №22

14 Ловец идей

Добавьте идею из поля в конец списка.



Пробелы по краям названия убираются. Поле очищается только после успешного добавления.

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Умеем брать строку из Entry.

Новое перед этой задачей

Listbox.insert(), порядок, дубликаты.

Цель решения

Добавлять каждую новую строку в конец видимого списка.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 14: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
box = tk.Listbox(root)
box.pack()

def add_item():
    text = entry.get()
    box.insert(tk.END, text)
```

Запомнить

`Listbox.insert()`, порядок, дубликаты.

Если условие не запрещает, одинаковые строки остаются разными элементами.

Перед кодом. Какой индекс удобнее использовать, чтобы всегда добавлять в конец?

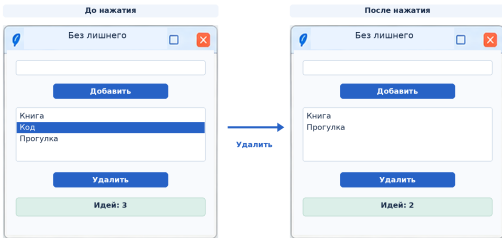
После ответа на вопрос — переходите в Контекст и решайте задачу 22. Следующую теорию показывайте только после попытки.

Задача 15: Без лишнего

Tkinter 15/20 · в вашем контексте №23

15 Без лишнего

Удаляется выбранная строка, а не случайная.



После удаления выделение снимается. Без выбора повторное удаление ничего не делает.

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Listbox уже умеем заполнять.

Новое перед этой задачей

`curselection()`, удаление по индексу.

Цель решения

Найти выбранную позицию и удалить именно её; корректно пережить пустой выбор.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 15: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def remove_selected():  
    selected = box.curselection()  
    if selected:  
        i = selected[0]  
        box.delete(i)
```

Запомнить

`curselection()`, удаление по индексу.

Не удаляйте по тексту: одинаковые строки могут встречаться несколько раз.

Перед кодом. Что возвращает `curselection()`, если ничего не выбрано?

После ответа на вопрос — переходите в Контекст и решайте задачу 23. Следующую теорию показывайте только после попытки.

Задача 16: Черновик под лупой

Tkinter 16/20 · в вашем контексте №24

16 Черновик под лупой

Считаем весь текст, включая введённые переводы строк.

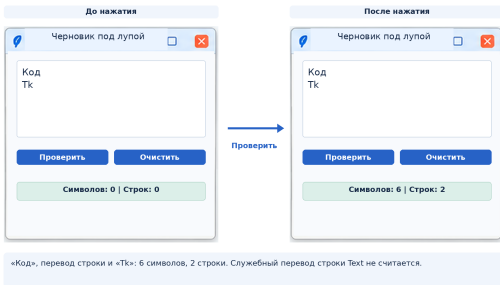


Схема интерфейса, не pixel-ный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Entry и Listbox уже знакомы.

Новое перед этой задачей

Text и индексы "1.0" / "end-1c".

Цель решения

Прочитать многострочный текст без служебного перевода строки Tkinter.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 16: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
editor = tk.Text(root, height=6, width=30)
editor.pack()
```

```
def read_text():
    text = editor.get("1.0", "end-1c")
    print(text)
```

Запомнить

Text и индексы "1.0" / "end-1c".

Text использует индекс «строка.столбец», а в конце хранит дополнительный .

Перед кодом. Почему часто читают до end-1c, а не просто до end?

После ответа на вопрос — переходите в Контекст и решайте задачу 24. Следующую теорию показывайте только после попытки.

Задача 17: Заказ для мастерской

Tkinter 17/20 · в вашем контексте №25

17 Заказ для мастерской

Один размер и две независимые дополнительные услуги.

До нажатия

После нажатия

Рассчитать

Параметры сохраняются после расчёта. Следующий клик не прибавляет стоимость повторно.

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Отдельно умеем радиокнопки и флажки.

Новое перед этой задачей
комбинация Radio + Checkbutton.

Цель решения

Снять текущие параметры формы и каждый раз пересчитать итог с нуля.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 17: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
def calculate():  
    total = 100 if size.get() == "small" else 160  
    if gift_wrap.get():  
        total += 20  
    if urgent.get():  
        total += 40
```

Запомнить

комбинация Radio + Checkbutton.

Повторный клик «Рассчитать» не должен накапливать старые надбавки.

Перед кодом. Какие части цены взаимоисключающие, а какие независимые?

После ответа на вопрос — переходите в Контекст и решайте задачу 25. Следующую теорию показывайте только после попытки.

Задача 18: Визитка участника

Tkinter 18/20 · в вашем контексте №26

18 Визитка участника

Соберите два поля и настройку связи в понятную форму.

The diagram illustrates the state of a Tkinter window titled 'Визитка участника' (Participant Card) before and after a button click. An arrow labeled 'Собрать карточку' (Collect card) points from the 'До нажатия' (Before click) state to the 'После нажатия' (After click) state.

До нажатия (Before click):

- Имя (Name): Input field with 'Аня' (Anya).
- Проект (Project): Input field with 'Мини-игра' (Mini-game).
- Открыт к сообщениям (Open to messages): Checked checkbox.
- Собрать карточку (Collect card): Blue button.
- Карточка не создана (Card not created): Green button.

После нажатия (After click):

- Имя (Name): Input field with 'Аня' (Anya).
- Проект (Project): Input field with 'Мини-игра' (Mini-game).
- Открыт к сообщениям (Open to messages): Checked checkbox.
- Собрать карточку (Collect card): Blue button.
- Аня / Мини-игра / связь: открыта (Anya / Mini-game / link: open): Green button.

Поля не очищаются. Пустое имя или проект приводит к сообщению об ошибке.

Схема интерфейса, не pixel-perfect эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Умеем собирать несколько типов виджетов.

Новое перед этой задачей

Frame и логическая группировка формы.

Цель решения

Разбить интерфейс на понятные блоки, не создавая второе окно.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 18: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
form = tk.Frame(root)
form.pack(padx=12, pady=12)

tk.Label(form, text="Имя").grid(row=0, column=0)
name = tk.Entry(form)
name.grid(row=0, column=1)
```

Запомнить

Frame и логическая группировка формы.

pack и grid можно сочетать только в разных родительских контейнерах.

Перед кодом. Чем Frame отличается от второго Tk()?

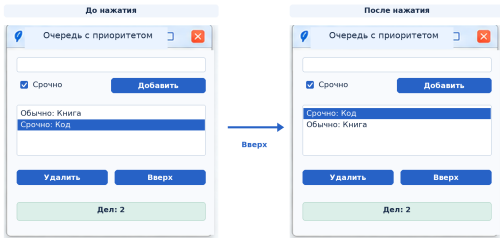
После ответа на вопрос — переходите в Контекст и решайте задачу 26. Следующую теорию показывайте только после попытки.

Задача 19: Очередь с приоритетом

Tkinter 19/20 · в вашем контексте №27

19 Очередь с приоритетом

Поднимите выбранное дело на одну позицию.



Выделение следует за перемещённой строкой. Количество дел остаётся прежним.

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Умеем читать выбранный индекс Listbox.

Новое перед этой задачей

перестановка элемента + восстановление выделения на её новый индекс.

Цель решения

Переместить выбранную строку и перенести выделение на её новый индекс.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 19: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
selected = box.curselection()
if selected:
    i = selected[0]
    if i > 0:
        text = box.get(i)
        box.delete(i)
        box.insert(i - 1, text)
        box.selection_set(i - 1)
```

Запомнить

перестановка элемента + восстановление выбора.

После перемещения «тот же элемент» уже имеет другой индекс.

Перед кодом. Что должно произойти с выделением после сдвига строки вверх?

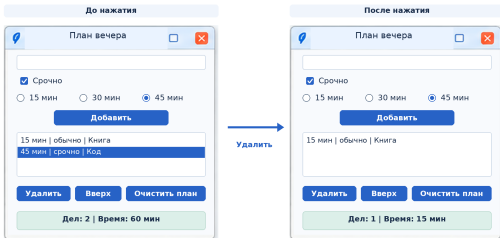
После ответа на вопрос — переходите в Контекст и решайте задачу 27. Следующую теорию показывайте только после попытки.

Задача 20: План вечера

Tkinter 20/20 · в вашем контексте №28

20 План вечера

Каждое дело хранит своё время и приоритет.



Удаляется дело на 45 минут. В плане остаётся 15 минут; настройки нового дела сохраняются.

Схема интерфейса, не пиксельный эталон. Цвета, шрифты и отступы можно выбирать.

Уже умеем

Все нужные виджеты уже знакомы.

Новое перед этой задачей

список записей как данные; сводка.

Цель решения

Хранить записи отдельно от Listbox и строить интерфейс как представление этих данных.

Не листайте дальше автоматически: сначала разберите только новый инструмент, затем дайте студентам попробовать задачу.

Перед задачей 20: только то, что сейчас нужно

Нейтральный пример: не решение задачи контекста

```
items = []

def add_item():
    record = {
        "name": name.get(),
        "minutes": int(minutes.get())
    }
    items.append(record)
```

Запомнить

список записей как данные; сводка.

Одинаковые названия могут быть разными записями; данные не должны жить только в тексте виджета.

Перед кодом. Какие данные нужно сохранить в момент добавления записи, чтобы позже посчитать сводку?

После ответа на вопрос — переходите в Контекст и решайте задачу 28. Следующую теорию показывайте только после попытки.

После двадцатой задачи

Виджеты

Tk, Label, Button, Entry, Checkbutton, Radiobutton, Scale, Listbox, Text, Frame.

Действия

command, get, config, delete/insert, выбор, удаление, перестановка.

Состояние

Обычные переменные, Tkinter-переменные, память, коллекции записей.

Главная архитектура

Интерфейс показывает данные; callback читает действие; состояние хранит то, что должно пережить один клик.

После этого уже можно переходить от учебных задач к небольшим самостоятельным оконным приложениям.